Podstawy uczenia maszynowego

metody jądrowe

ćwiczenia nr 3

Akademia Górniczo-Hutnicza
im. Stanisława Staszica w Krakowie
AGH University of Krakow

semestr letni 2025/2026

Paweł Szczerbiak

Czym się zajmiemy na ćwiczeniach?

Ćwiczenia 1: metody liniowe

Ćwiczenia 2: metody oparte na sąsiedztwie

Ćwiczenia 3: metody jądrowe

Ćwiczenia 4: metody probabilistyczne

Ćwiczenia 5: klasyfikacja niezbalansowana

Ćwiczenia 6: selekcja cech i redukcja wymiarowości

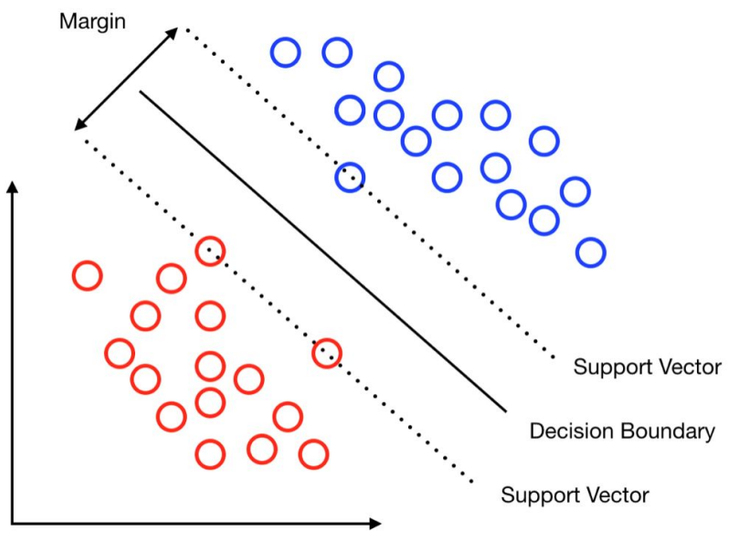
Ćwiczenia 7: klasteryzacja

Zagadnienia

1. Klasyfikator największego marginesu (MMC)
2. Liniowa maszyna wektorów nośnych (*linear* SVM)
3. Dodatnio określona funkcja jądra
4. Sztuczka z jądrem (*kernel trick*)
5. Jądrowa maszyna wektorów nośnych (*kernel* SVM)
6. Podstawowe funkcje jądrowe (*kernels*)

Klasyfikator największego marginesu (MMC)

1. **Hiperpłaszczyzna**: granica decyzyjna używana do oddzielania punktów z różnych klas w przestrzeni cech
2. **Wektory nośne**: wektory równoległe do hiperpłaszczyzny, przechodzące przez najbliższe punkty z danej klasy
3. **Margines**: odległość między wektorami nośnymi
4. **Klasyfikator największego marginesu** (*maximum margin classifier, hard margin classifier*): klasyfikator **binarny** separujący klasy przy zachowaniu największego marginesu



Uwaga: model nie podlega treningowi - parametry są *implicite* zawarte w danych treningowych (*lazy learning*, *instance-based learning*) - por. algorytm kNN

Klasyfikator największego marginesu (MMC)

1. Rozważmy problem optymalizacyjny:

dane wejściowe (x_i, y_i)

$$\max_{\beta_0,\beta_1,\ldots,\beta_p} \quad M$$ (margines)

$$\text{subject to} \sum_{j=1}^{p} \beta_j^2 = 1,$$ (współczynniki modelu)

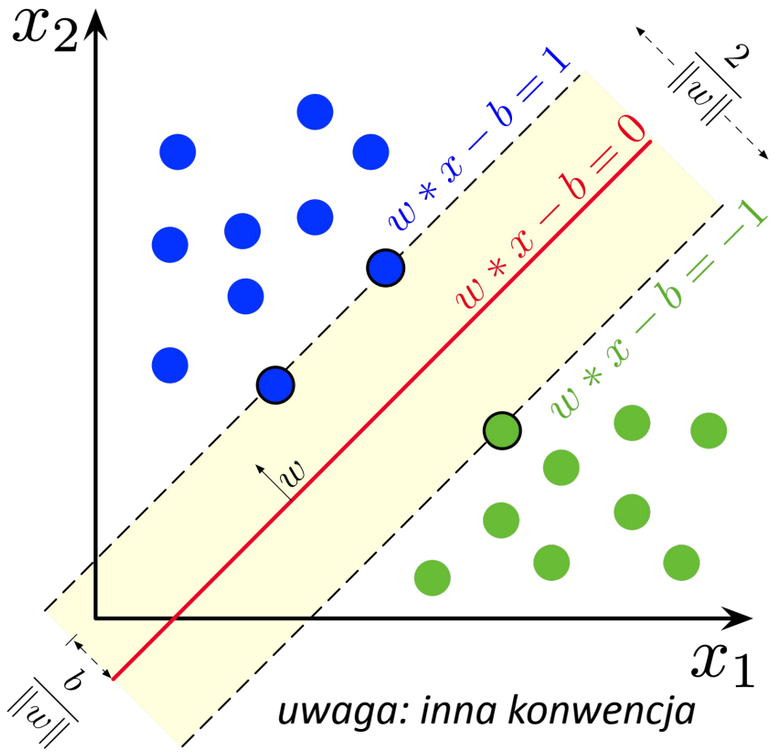
$$y_i(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}) \geq M \quad \text{for all } i = 1, \ldots, n$$

2. Definiujemy: $w = (w_1, \ldots, w_p) = \beta / M$

$$\min_{\beta_0, w} \quad \frac{1}{2}\|w\|^2$$ (nowe współczynniki modelu)

$$\text{subject to}$$

$$y_i(\beta_0 + w \cdot x_i) \geq 1 \quad \text{for all } i = 1, \ldots, n$$



wikipedia.org
web.stanford.edu/class/stats202

Klasyfikator największego marginesu (MMC)

3. Używamy mnożników Karusha-Kuhna-Tuckera (KKT) **- problem dualny**

$$\max_{\alpha} \min_{\beta_0, w} \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i [y_i(\beta_0 + w \cdot x_i) - 1]$$

subject to $\alpha_i \geq 0$.

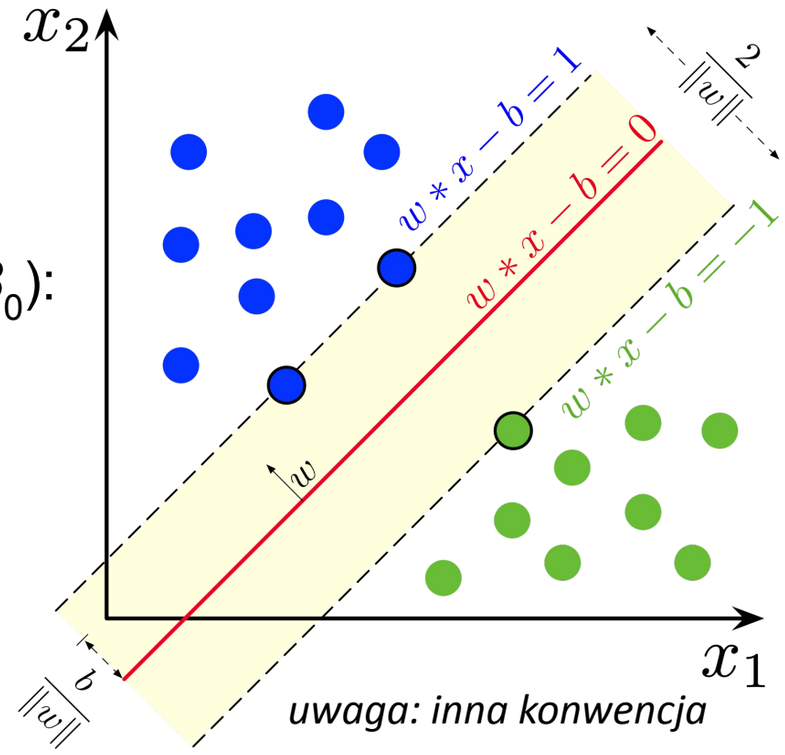
4. Obliczamy pochodne cząstkowe względem parametrów (w_i, β_0):

$\hat{w} = \sum_{i=1}^{n} \alpha_i y_i x_i$, and $\sum_{i=1}^{n} \alpha_i y_i = 0$

5. Podstawiamy rozwiązanie wyżej do pkt. 3:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{i'=1}^{n} \alpha_i \alpha_{i'} y_i y_{i'} (x_i \cdot x_{i'})$$

subject to $\alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0.$



wikipedia.org
web.stanford.edu/class/stats202

Klasyfikator największego marginesu (MMC)

6. Konsekwencją warunku KKT jest to, że tylko te mnożniki są niezerowe (α_i), dla których spełnione jest:

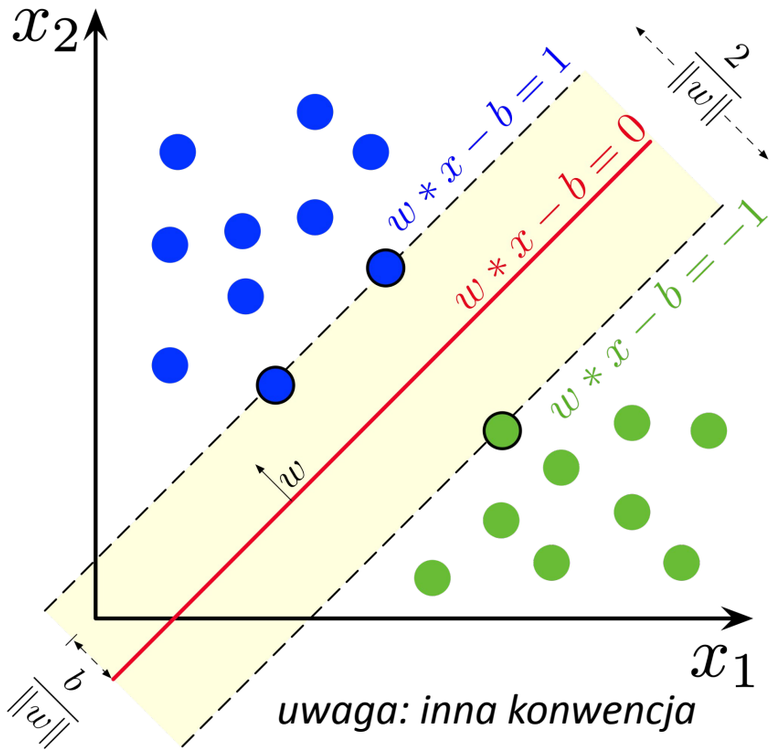
$$y_i(\beta_0 + w \cdot x_i) = 1$$ (tzn. tylko punkty leżące na marginesie)

Obserwacje:

- Rozwiązanie (α_i) zależy tylko od najbliższych punktów
- Warunek optymalizacyjny zależy tylko od:
  - iloczynu $\alpha_i\ \alpha_j$ – można przyspieszyć trening
  - iloczynu skalarnego $x_i\ x_j$ – można użyć nieliniowej funkcji jądra (o tym dalej)

Problem:

- Czasem punkty **nie są idealnie** liniowo separowalne

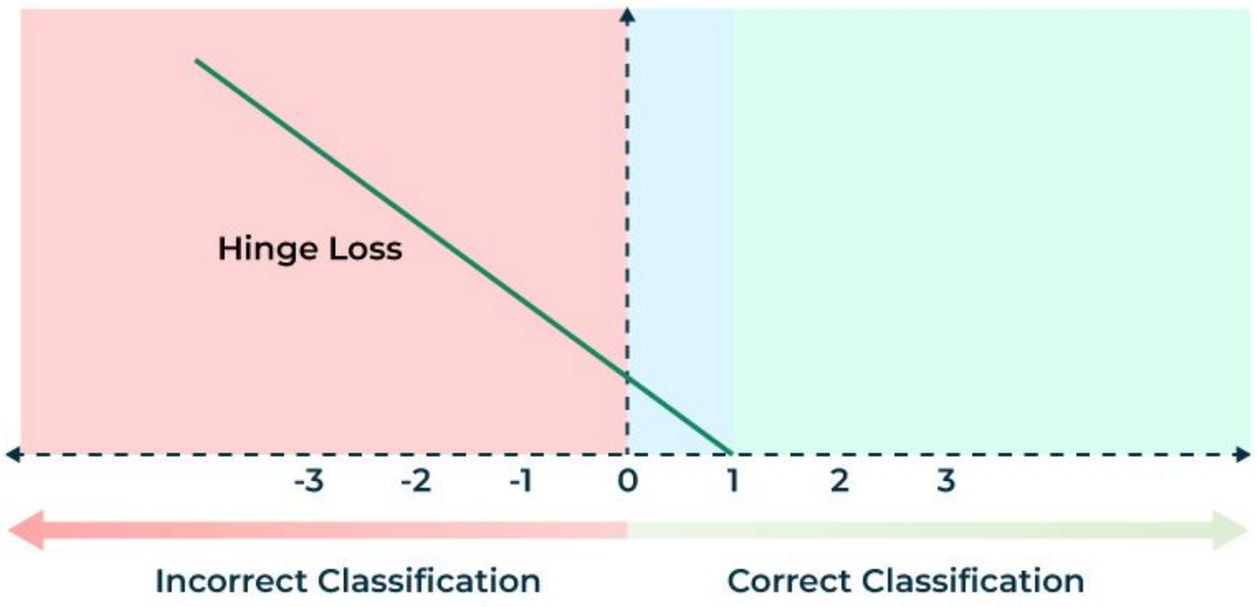


wikipedia.org
web.stanford.edu/class/stats202

Liniowa maszyna wektorów nośnych (*linear* SVM)

1. Maszyna wektorów nośnych (*support vector machine, support vector classifier, soft margin classifier*) dokonuje **relaksacji warunku** z MMC
2. Pewna liczba punktów może znajdować się po złej stronie marginesu lub nawet hiperpłaszczyzny, dopuszczając “luz” dla każdego przypadku
3. W tym celu używamy **zawiasowej funkcji kosztu** (*hinge loss*)

$$\max\left(0, 1 - y_i\left(\mathbf{w}^\mathsf{T}\mathbf{x}_i - b\right)\right)$$



geeksforgeeks.org

Liniowa maszyna wektorów nośnych (*linear* SVM)

1. Rozważmy problem optymalizacji:

$$\max_{\beta_0,\beta,\epsilon} \quad M$$

$$\text{subject to} \sum_{j=1}^{p} \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}) \geq M(1-\epsilon_i)$$

$$\epsilon_i \geq 0 \text{ for all } i = 1, \ldots, n, \quad \sum_{i=1}^{n} \epsilon_i \leq C.$$

(zmienne luzu, *slack*) (budżet, *budget*)

- *C* = 0 redukuje SVM do MMC
- jeśli *C* jest zbyt małe, mamy *overfitting*
- jeśli *C* jest zbyt duże, mamy gorszą separowalność

2. Definiujemy: $w = (w_1, \ldots, w_p) = \beta/M$

(L2) (*hinge loss*)

$$\min_{\beta_0,w,\epsilon} \quad \frac{1}{2}\|w\|^2 + D\sum_{i=1}^{n} \epsilon_i$$

$$\text{subject to}$$

$$y_i(\beta_0 + w \cdot x_i) \geq (1-\epsilon_i) \quad \text{for all } i = 1, \ldots, n,$$

$$\epsilon_i \geq 0 \quad \text{for all } i = 1, \ldots, n.$$

- *D* (kara, *penalty)* jest odwrotnie proporcjonalna do *C*

web.stanford.edu/class/stats202

Liniowa maszyna wektorów nośnych (*linear* SVM)

3. Używamy mnożników Karusha-Kuhna-Tuckera (KKT) **- problem dualny**

$$\max_{\alpha,\mu} \min_{\beta_0,w,\epsilon} \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i[y_i(\beta_0 + w \cdot x_i) - 1 + \epsilon_i] + \sum_{i=1}^{n}(D - \mu_i)\epsilon_i$$

$$\text{subject to} \quad \alpha_i \geq 0, \mu_i \geq 0, \qquad \text{for all } i = 1, \ldots, n.$$

4. Obliczamy pochodne cząstkowe względem parametrów (w_i, β_0, ϵ_i):

$$\hat{w} = \sum_{i=1}^{n} \alpha_i y_i x_i, \quad \sum_{i=1}^{n} \alpha_i y_i = 0, \quad \mu_i = D - \alpha_i$$

web.stanford.edu/class/stats202

Liniowa maszyna wektorów nośnych (*linear* SVM)

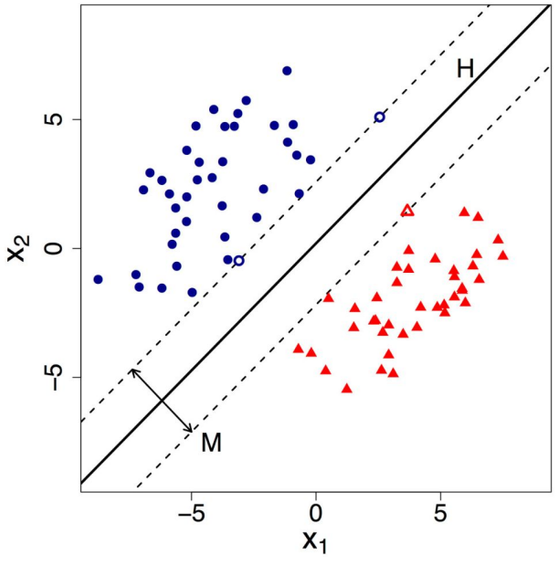
5. Podstawiamy rozwiązanie wyżej do pkt. 3:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{i'=1}^{n} \alpha_i \alpha_{i'} y_i y_{i'} (x_i \cdot x_{i'})$$
$$\text{subject to} \quad 0 \le \alpha_i \le D \ \text{ for all } i = 1, \ldots, n,$$
$$\sum_i \alpha_i y_i = 0.$$

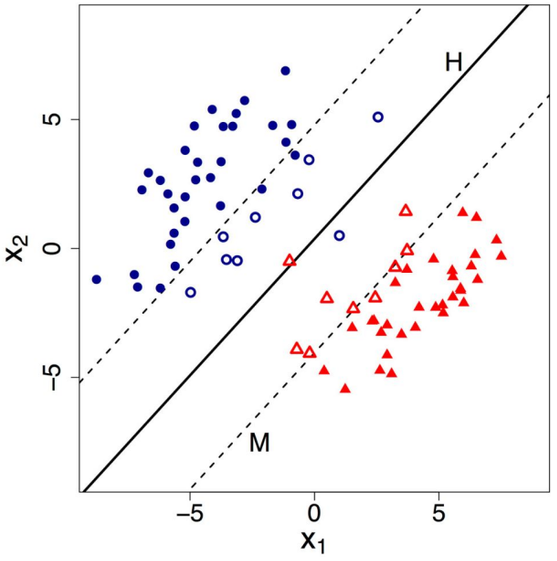
6. Konsekwencją warunku KKT jest to, że tylko te mnożniki są niezerowe (α_i), dla których spełnione jest (**punkty wchodzą za margines**):

$$y_i(\beta_0 + w \cdot x_i) \le 1,$$

hard ($C = 0$)



soft ($C > 0$)



Problem: czasem punkty **nie są w ogóle** liniowo separowalne

surveypractice.org
web.stanford.edu/class/stats202

Dodatnio określona funkcja jądra

1. Rozważmy symetryczną funkcję ciągłą: $K : [a, b] \times [a, b] \to \mathbb{R}$
2. Funkcję tę nazywamy dodatnio określony jądrem, gdy:

$$\sum_{i=1}^{n} \sum_{j=1}^{n} K(x_i, x_j) c_i c_j \geq 0$$

[uogólnienie pojęcia macierzy dodatnio **półokreślonej**]

dla dowolnych sekwencji punktów $x_1 \ldots x_n$ oraz liczb rzeczywistych $c_1 \ldots c_n$.

3. **Twierdzenie Mercera:** dowolną funkcję rzeczywistą $K(x, y)$ możemy rozłożyć jako nieskończoną, zbieżną sumę kombinacji liniowej iloczynu funkcji ortonormalnych, jeśli spełnione jest:

$$\iint g(x) K(x, y) g(y) \, dx \, dy \geq 0$$

[uogólnienie, że każda macierz dodatnio **półokreślona** jest **macierzą Grama**]

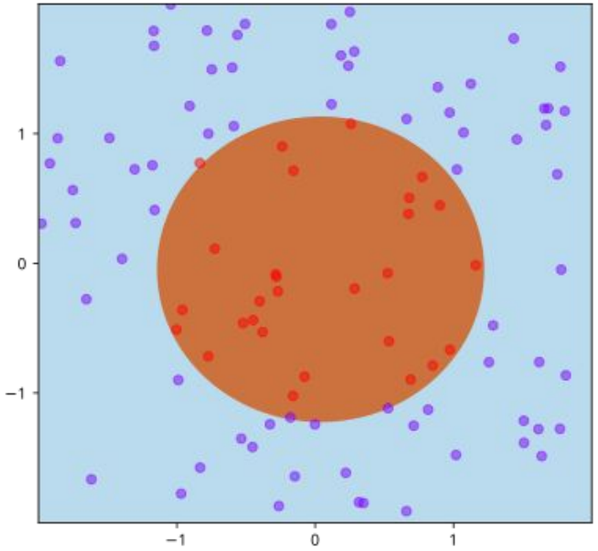
dla dowolnych funkcji $g(x)$ z przestrzeni funkcji lokalnie całkowalnych z kwadratem

Sztuczka z jądrem (*kernel trick*)

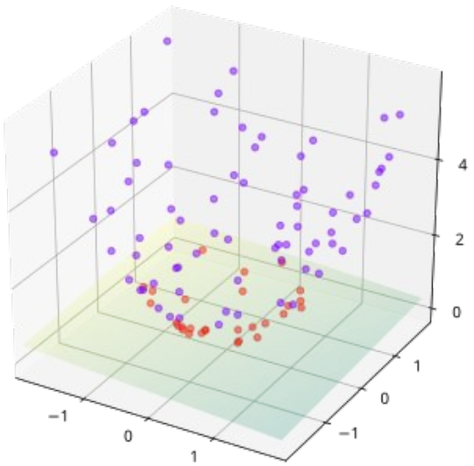
1. **Twierdzenie Covera:** problem klasyfikacji zrzutowany po **nieliniowym** przekształceniu do wyżej wymiarowej przestrzeni jest liniowo separowalny z większym prawdopodobieństwem
2. **Sztuczka z jądrem:** zakładamy, że istnieje ta wyżej wymiarowa przestrzeń z dobrze zdefiniowanym iloczynem skalarnym, ale nie musimy jej jawnie definiować
3. **Zaleta:** nie tworzymy nowych wektorów cech i nie liczymy ich iloczynów skalarnych!
4. Jeśli **jądro spełnia warunek Mercera**, to istnieje takie nieliniowe przekształcenie $\varphi\colon \mathcal{X} \to \mathcal{V}$ spełniające:

$$k(x_i, x_{i'}) = \langle \varphi(x_i), \varphi(x_{i'}) \rangle_{\mathcal{V}}$$

2D



3D



wikipedia.org

Jądrowa maszyna wektorów nośnych (*kernel* SVM)

1. Dokonujemy podstawienia:

$$(x_i \cdot x_{i'}) \longrightarrow k(x_i, x_{i'})$$ i tyle…

2. Podstawowe funkcje jądrowe (*kernels*):

liniowe $k(x_i, x_{i'}) = x_i^T x_{i'}$

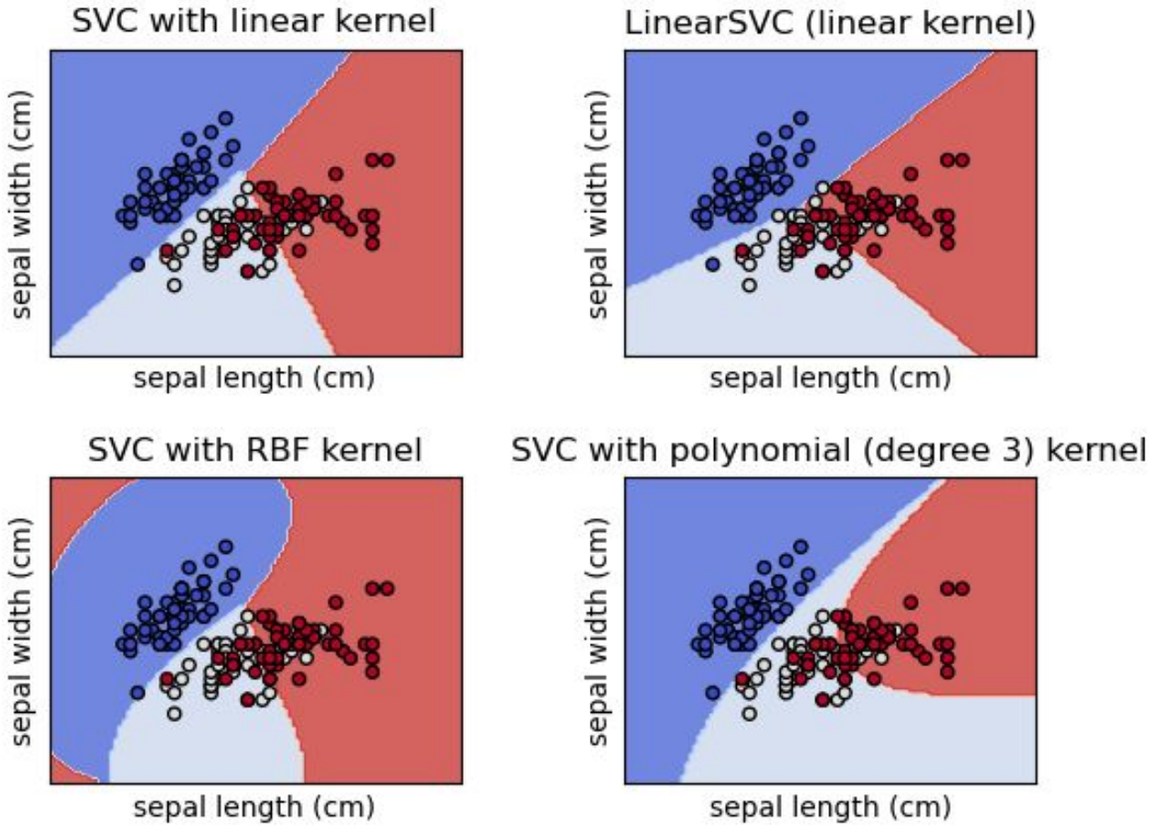
wielomianowe $k(x_i, x_{i'}) = (x_i^T x_{i'} + c)^p$

RBF $k(x_i, x_{i'}) = \exp\left(-\frac{||x_i - x_{i'}||^2}{\gamma}\right)$

(*radial basis function*)

Gaussowskie $\gamma = 2\sigma^2$

(*jak wyżej, gdzie σ to wariancja cech)*



UWAGA: forma przekształcenia jest tutaj znana, ale skomplikowana

scikit-learn.org

Do zapamiętania

1. Klasyfikator największego marginesu (MMC)
1. Uczenie oparte na przykładach (*instance-based learning*) + przykłady
2. Problem pierwotny a problem dualny
3. Liniowa maszyna wektorów nośnych (*linear* SVM)
4. Zawiasowa funkcja kosztu (*hinge loss*)
5. Dodatnio określona funkcja jądra
6. Sztuczka z jądrem (*kernel trick*)
7. Jądrowa maszyna wektorów nośnych (*kernel* SVM)
8. Podstawowe funkcje jądrowe (*kernels*)